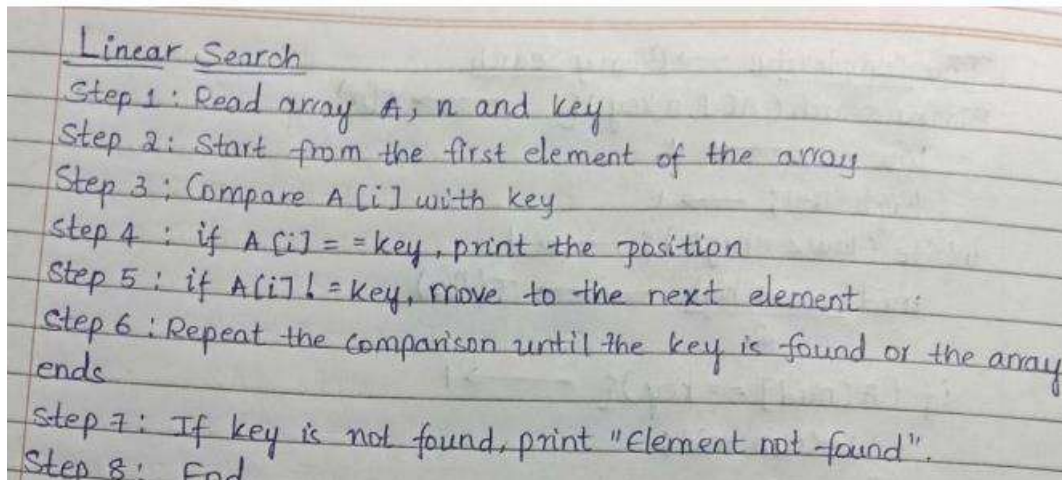


Practical-2

Aim: To Implementation and Time analysis of linear and binary search algorithm.

1) Linear search

Algorithm:



Program:

```
#include <iostream>

using namespace std;

class LinearSearch
{
public:
    int arr[100], n, key;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
```

Name: V. Bhavya sri
Enrollment no: 92460118227

```
for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}

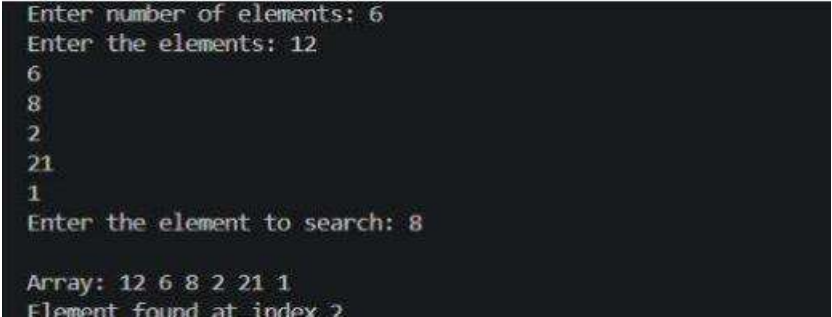
cout << "Enter the element to search: ";
cin >> key;
}

void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void search()
{
    int i;
    for (i = 0; i < n; i++)
    {
        if (arr[i] == key)
        {
            cout << "Element found at index " << i << endl;
            break;
        }
    }
}
```

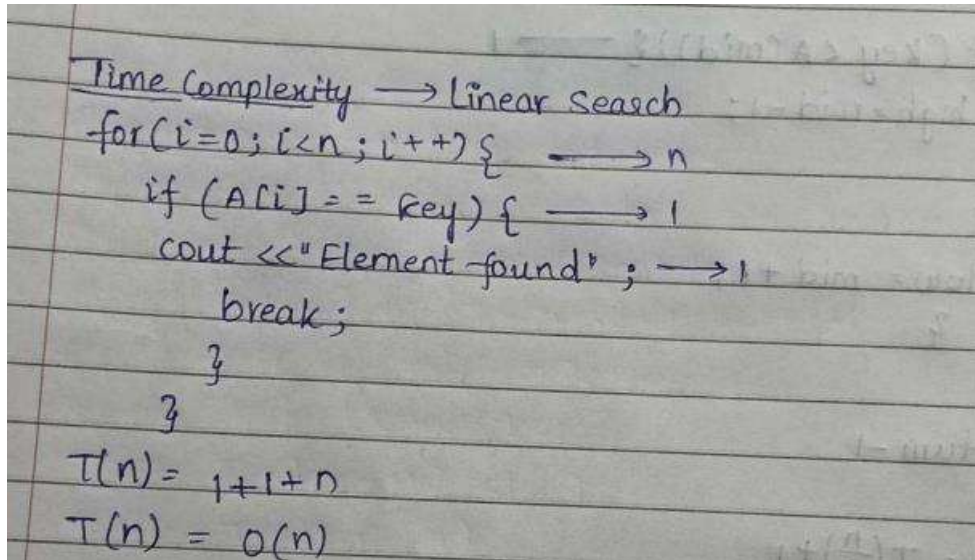
```
    }  
    if (i == n)  
    {  
        cout << "Element not found." << endl;  
    }  
}  
};  
int main()  
{  
    LinearSearch ls;  
    ls.arrayInput();  
    cout << "\nArray: ";  
    ls.printArray();  
    ls.search();  
    return 0;  
}
```

Output:



```
Enter number of elements: 6  
Enter the elements: 12  
6  
8  
2  
21  
1  
Enter the element to search: 8  
  
Array: 12 6 8 2 21 1  
Element found at index 2
```

Time complexity:



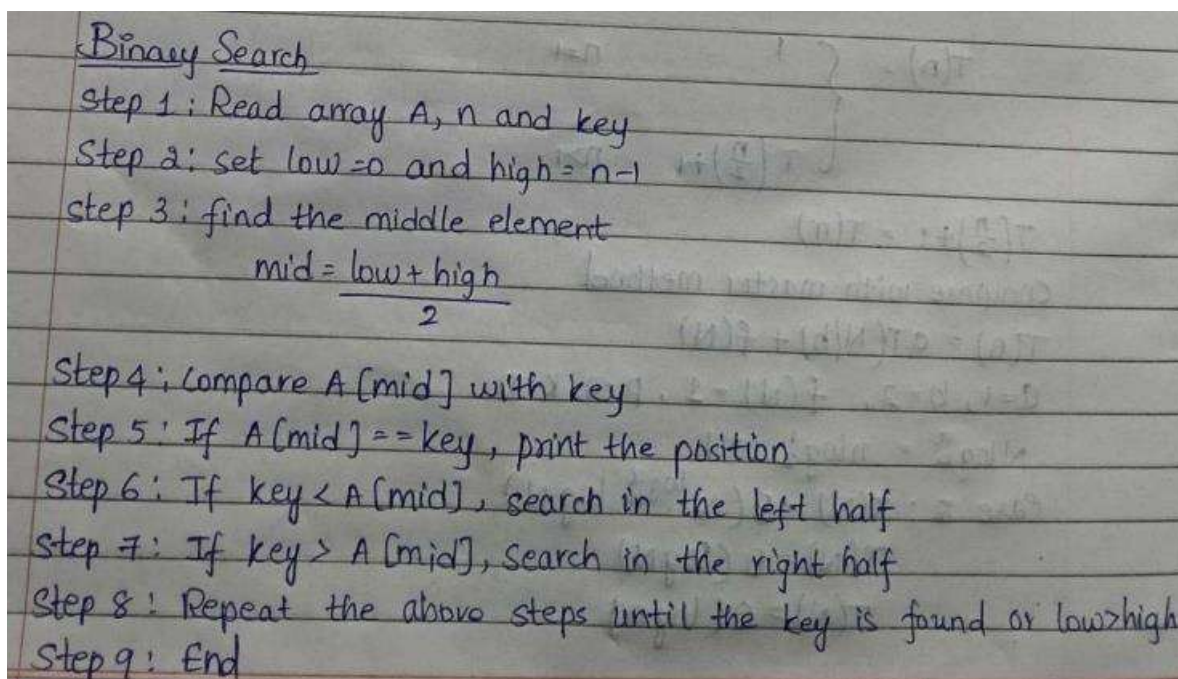
Time Complexity $\rightarrow$ Linear Search

```
for (i=0; i<n; i++) {  $\rightarrow n$ 
    if (A[i] == key) {  $\rightarrow 1$ 
        cout << "Element found" ;  $\rightarrow 1$ 
        break;
    }
}
```

$T(n) = 1 + 1 + n$
 $T(n) = O(n)$

2) Binary search

Algorithm:



Binary Search

Step 1: Read array A, n and key

Step 2: set low=0 and high = n-1

Step 3: find the middle element
$$\text{mid} = \frac{\text{low} + \text{high}}{2}$$

Step 4: Compare A[mid] with key

Step 5: If A[mid] == key, print the position

Step 6: If key < A[mid], search in the left half

Step 7: If key > A[mid], search in the right half

Step 8: Repeat the above steps until the key is found or low > high

Step 9: End

Program:

```
#include <iostream>

using namespace std;

class BinarySearch
{
public:
    int arr[100], n, key;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the sorted elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
        cout << "Enter the element to search: ";
        cin >> key;
    }
    void printArray()
    {
        for (int i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }
    }
}
```

Name:V.Bhavya sri
Enrollment no: 92460118227

```
        cout << endl;
    }
    void search()
    {
        int low = 0;
        int high = n - 1;
        while (low <= high)
        {
            int mid = (low + high) / 2;
            if (arr[mid] == key)
            {
                cout << "Element found at index " << mid << endl;
                return;
            }
            else if (key < arr[mid])
            {
                high = mid - 1;
            }
            else
            {
                low = mid + 1;
            }
        }
        cout << "Element not found." << endl;
    }
```

Name:V.Bhavya sri
Enrollment no: 92460118227

```
};  
  
int main()  
{  
    BinarySearch bs;  
    bs.arrayInput();  
    cout << "\nArray: ";  
    bs.printArray();  
    bs.search();  
    return 0;  
}
```

Output:

```
Enter number of elements: 6  
Enter the sorted elements: 6  
7  
8  
9  
12  
22  
Enter the element to search: 9  
  
Array: 6 7 8 9 12 22  
Element found at index 3
```

Time complexity:

Time Complexity: $\rightarrow$ Binary Search

```

Binary search (A[], n, key) {  $\rightarrow T(n)$ 
    low = 0;  $\rightarrow 1$ 
    high = n-1;  $\rightarrow 1$ 
    while (low <= high) {  $\rightarrow 1$ 
        mid =  $\frac{low + high}{2}$   $\rightarrow T(n/2)$ 

        if (A[mid] == key) {  $\rightarrow 1$ 
            return mid;
        }
        else if (key < A[mid]) {  $\rightarrow 1$ 
            high = mid - 1;
        }
        else {
            low = mid + 1;
        }
    }
    return -1
}

T(n) = T( $\frac{n}{2}$ ) + 1

T(n) =  $\begin{cases} 1 & n=1 \\ T(\frac{n}{2}) + 1 & n>1 \end{cases}$ 

```

$T(\frac{n}{2}) + 1 = T(n)$
 Compare with master method
 $T(n) = aT(N/b) + f(N)$
 $a=1, b=2, f(N)=1, p=0, k=0$
 $N \log_2^a = n \log_2^1 = 1$
 Case 2: $T(N) = \Theta(N^{\log_2^a} \log N)$
 $T(N) = \Theta(\log N)$

Conclusion :

The implementation and time analysis of Linear Search and Binary Search helps us understand their efficiency. Linear Search is simple and can work on any array, but it may require **$O(n)$** comparisons. Binary Search is faster with **$O(\log n)$** time in average and worst cases, but it requires the array to be sorted. Therefore, Binary Search is more efficient for searching large sorted datasets.